I. INTRODUCTION

Software delivery has changed significantly over the last decade. Applications are updated more frequently, release cycles are shorter, and users expect stable services even when new features are introduced at a fast pace. Because of this, manual build and deployment practices are no longer enough for most modern projects. Continuous Integration and Continuous Deployment, commonly grouped as CI/CD, help teams automate repetitive work such as pulling code, installing dependencies, running tests, building images, and deploying updates [1].

In practice, a CI/CD pipeline is not just a technical convenience. It shapes how quickly a team can respond to defects, how confidently changes can be shipped, and how much effort is required to maintain a release process. A strong pipeline can reduce human error and improve traceability, but it can also introduce noticeable overhead if the chosen toolchain is poorly matched to the project.

Several CI platforms are now common in both academic and industrial settings. Jenkins remains popular because of its mature plugin ecosystem and the control it gives to teams that manage their own infrastructure. GitHub Actions has become attractive for teams already working in GitHub because workflow configuration is easy to connect with repositories. GitLab CI offers a similarly integrated experience for GitLab-based projects. Although all three tools support automated testing and deployment, their real-world behavior differs in terms of execution speed, configuration effort, and consistency.

One problem in existing comparisons is that benchmark conditions often vary. Different hardware, different applications, and different test cases make it hard to draw fair conclusions. Some studies evaluate usability, while others focus only on one stage of the pipeline. As a result, there is still value in a controlled, practical comparison that keeps the application, environment, and testing process constant.

This paper addresses that need. It evaluates multiple CI tools under the same conditions by using a containerized Flask application and a fixed test suite. The main goal is to understand how these toolchains behave when they perform the same tasks in the same environment. The outcome is intended to help students, researchers, and practitioners choose tools based on measurable trade-offs rather than on popularity alone.

II. LITERATURE REVIEW

Research on DevOps and CI/CD has consistently shown that automation improves delivery reliability and shortens feedback cycles. Humble and Farley described continuous delivery as a disciplined way to move software changes toward release by automating build, test, and deployment activities [1]. Their work remains a core reference because it explains why repeatable automation matters for stable software delivery.

Bass, Weber, and Zhu later examined DevOps from an architectural viewpoint and argued that software quality is strongly influenced by coordination between development and operations teams [2]. Their discussion makes it clear that tools alone do not solve delivery problems; architecture, monitoring, and deployment practice also affect pipeline efficiency.

Recent discussions around high-performing software teams, especially in performance-focused DevOps literature, stress the importance of fast feedback and low-friction deployment workflows [3]. These ideas support the use of CI/CD not only as an automation layer but also as a way to improve engineering decisions.

Comparisons of Jenkins, GitHub Actions, and GitLab CI often highlight usability and ecosystem fit. Jenkins is usually recognized for flexibility, plugin support, and deep customization. At the same time, its setup and maintenance effort can be high, especially when teams host and manage the infrastructure themselves [7]. GitHub Actions and GitLab CI reduce some of that effort by offering cloud-hosted runners and repository-level integration [8], [9]. However, those conveniences may come with queueing delays, network dependence, and less direct visibility into system resources.

Testing frameworks are also an important part of the pipeline. PyTest is widely used for unit and API testing because it is lightweight and easy to maintain [12]. Selenium is useful for browser-based automation and end-to-end testing, but its execution cost is naturally higher because it depends on browser startup and UI interaction [11]. Cypress has also become relevant in modern test automation, but its use depends heavily on the application stack and frontend environment [13].

Containerization has made benchmarking more reliable because Docker can package an application and its dependencies into a reproducible environment [6]. This reduces the classic "works on my machine" issue and makes tool comparisons more fair. Still, container build time and image management can add non-trivial overhead to a pipeline, especially in constrained environments.

The literature therefore suggests two clear points. First, CI/CD tools must be judged in context rather than in isolation. Second, fair evaluation requires a controlled setup. This paper builds on those ideas by combining multiple pipeline stages into a single practical comparison.

III. METHODOLOGY

A. Experimental Design

The study was carried out in a controlled Linux-based virtual machine so that all tools operated under the same hardware conditions. CPU allocation, memory, and storage were kept fixed during the entire experiment. No changes were made to the host setup once benchmarking began. This helped reduce performance variation caused by environmental differences [5].

Each CI tool was executed repeatedly under the same workflow. Average values were used for reporting so that temporary fluctuations did not dominate the results. The same repository, application files, and test suite were used for all runs.

B. Benchmark Application

A small Flask web application served as the benchmark system. The application included several API routes for normal responses, data access, and controlled error cases. This allowed the pipeline to cover multiple situations rather than only a single success path. The application was containerized with Docker so that every tool built and tested the same image each time.

C. Test Suite Standardization

To keep the comparison fair, the same set of nine tests was executed in every pipeline run. These tests checked route accessibility, expected response values, and basic failure handling. The suite remained unchanged throughout the experiment. That decision was important because even small differences in test design can distort the comparison between CI tools [6].

D. Pipeline Structure

The workflow for each platform followed the same basic order: pull the latest source code, install project dependencies, execute the test suite, and build the Docker image. The syntax of each tool was different, but the logic stayed the same. This made it possible to compare execution behavior without changing the actual work performed by the pipeline.

E. Evaluation Metrics

The tools were compared using the following metrics: pipeline execution time, Docker build time, test execution time, resource utilization where measurable, integration complexity, reliability across repeated runs, and compatibility with surrounding tools and environments. Together, these metrics provide a more complete picture than execution speed alone. They reflect both technical efficiency and day-to-day usability.

IV. EXPERIMENTAL SETUP

All experiments were performed inside a Linux virtual machine configured to remain unchanged during testing. The environment used Ubuntu 22.04/24.04, 8 GB RAM, 4 processors, and 60 GB of storage. This virtual machine acted as the common platform for running local pipelines, building containers, and collecting measurements.

The benchmark application was a Dockerized Flask service. The same container image definition was reused across all tools to avoid any shift in dependencies or build logic. The deployment process followed the usual steps of building the image and running the container on the target port.

For CI implementation, three tools were configured: Jenkins with a declarative local pipeline, GitHub Actions with a YAML workflow on cloud-hosted runners, and GitLab CI with a .gitlab-ci.yml configuration on shared runners. Each pipeline executed the same functional steps. Only the platform-specific syntax was different. This made the comparison more credible because the tools were solving the same problem in almost the same way.

To understand the cost of automation, a baseline was also recorded by running the application and tests manually without CI orchestration. That baseline provided a reference point for measuring how much extra time the full pipeline introduced [10].

Performance values were collected through pipeline logs and, for local execution, basic system monitoring tools. Jenkins allowed direct observation of CPU and memory usage because it ran on the local machine. Cloud platforms did not expose the same level of resource detail, so their evaluation relied more heavily on execution time and consistency across repeated runs.

V. RESULTS AND DISCUSSION

A. CI Tool Performance

The results show clear differences among the three CI tools. Jenkins delivered the fastest overall pipeline execution, with an average time of about 18.2 seconds. Its Docker build took roughly 8.8 seconds, while test execution remained close to 0.10 seconds. Because Jenkins ran locally, it benefited from direct access to system resources and predictable execution conditions. At the same time, it placed a visible load on the host machine, with average CPU utilization near 76.8% and memory usage around 780 MB.

GitHub Actions recorded an average pipeline execution time of 21.4 seconds. This was slightly slower than Jenkins, largely because the workflow ran in a remote environment. However, its Docker build performance was stronger at about 4.16 seconds. The result suggests that managed cloud infrastructure and caching can improve parts of the workflow even when total pipeline time is still affected by remote execution overhead [8].

GitLab CI showed the longest average pipeline duration at about 90.4 seconds. The test stage remained short, and build time was not dramatically worse than the other tools, but the total pipeline suffered because of shared runner delay, startup latency, and variability in the cloud environment [9]. This made GitLab CI the least predictable of the three in this experiment.

B. Testing Framework Impact

The testing framework also influenced the overall efficiency of the pipeline. PyTest completed its work in roughly 0.30 seconds and introduced very little overhead. Its lightweight design makes it a strong choice for backend and API testing, especially when fast feedback is important [12].

Selenium took longer, averaging around 0.91 seconds. The difference is expected because Selenium must initialize a browser session and interact with the UI layer. Although the increase may appear small in isolation, browser-driven testing becomes much more expensive as the number of test cases grows. This confirms that teams should choose the testing framework according to test purpose rather than assuming that one framework is best for every layer of the application [11].

C. Stability and Overhead

Repeated runs showed that Jenkins and GitHub Actions were relatively stable. Their execution times stayed within a narrow range, which is valuable in practical CI/CD work because predictable timing helps teams estimate feedback loops and release windows. GitLab CI displayed more variation, likely because shared infrastructure introduces scheduling and startup uncertainty.

Another important finding is the gap between baseline execution and automated pipeline execution. Manual execution of the application and tests took only about 0.34 seconds, while CI pipelines extended this into several seconds or, in the case of GitLab CI, over a minute. This overhead comes from dependency setup, workflow orchestration, container handling, and remote execution steps. In other words, automation brings reliability and repeatability, but it is never free [14].

D. Practical Interpretation

The results support a balanced view rather than a simple ranking. Jenkins is a strong option when a team wants speed, full control, and direct resource visibility. Its downside is the

effort required to configure and maintain it. GitHub Actions is well suited to teams that value convenience, repository integration, and reasonable performance without maintaining their own CI server. GitLab CI remains useful for GitLab-centered workflows, but in this experiment it was more affected by shared runner limitations than the other platforms.

VI. CONCLUSION

This paper examined the behavior of major CI/CD tools in a controlled environment using the same Flask application, Docker image, and test suite. The comparison showed that the three platforms all support automation effectively, but they do so with different trade-offs.

Jenkins produced the fastest pipeline in this study and offered the greatest level of control. That makes it valuable for teams that manage their own infrastructure and care about predictable execution. GitHub Actions provided a cleaner setup experience and strong build performance, making it a practical option for teams already working in the GitHub ecosystem. GitLab CI supported the same workflow but showed more delay and variation because of shared cloud resources.

The testing comparison also reinforced a common but important point: lightweight frameworks such as PyTest are more efficient for backend checks, while browser-based tools such as Selenium are better reserved for cases where end-to-end validation is necessary. Choosing the right testing tool therefore matters just as much as choosing the CI platform itself.

Another major conclusion is that CI/CD automation always introduces overhead when compared with manual execution. Even so, that overhead often remains acceptable because automation improves repeatability, reduces manual error, and supports more consistent delivery practices. The right decision is therefore not about removing overhead entirely, but about selecting a toolchain whose overhead is justified by its operational benefits [15].

Future work could extend this study by including more CI platforms, larger applications, cost analysis, and security-focused evaluation. It would also be useful to study hybrid setups in which local and cloud-based workflows are combined to balance speed, control, and scalability.

REFERENCES

[1] J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston, MA, USA: Addison-Wesley, 2010.

[2] L. Bass, I. Weber, and L. Zhu, DevOps: A Software Architect's Perspective. Boston, MA, USA: Addison-Wesley, 2015.

[3] N. Forsgren, J. Humble, and G. Kim, Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations. Portland, OR, USA: IT Revolution Press, 2018.

[4] G. Kim, K. Behr, and G. Spafford, The Phoenix Project: A Novel About IT, DevOps, and Helping Your Business Win, 5th ed. Portland, OR, USA: IT Revolution Press, 2018.

[5] M. Fowler, "Continuous Integration," ThoughtWorks, 2006. [Online]. Available: https://martinfowler.com/articles/continuousIntegration.html

[6] Docker Inc., "Docker Documentation," 2024. [Online]. Available: https://docs.docker.com

[7] Jenkins Project, "Jenkins User Documentation," 2024. [Online]. Available: https://www.jenkins.io/doc/

[8] GitHub Inc., "GitHub Actions Documentation," 2024. [Online]. Available: https://docs.github.com/actions

[9] GitLab Inc., "GitLab CI/CD Documentation," 2024. [Online]. Available: https://docs.gitlab.com/ee/ci/

[10] B. Behl and S. Behl, "Cybersecurity and Cyberwar: What Everyone Needs to Know," Oxford University Press, 2017.

[11] SeleniumHQ, "Selenium Documentation," 2024. [Online]. Available: https://www.selenium.dev/documentation/

[12] PyTest Development Team, "PyTest Documentation," 2024. [Online]. Available: https://docs.pytest.org

[13] Cypress.io, "Cypress Documentation," 2024. [Online]. Available: https://docs.cypress.io

[14] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams, "A Systematic Mapping Study of Infrastructure as Code Research," IEEE Trans. Softw. Eng., vol. 48, no. 1, pp. 185-208, 2022.

[15] S. M. Erich, C. Amrit, and M. Daneva, "A Qualitative Study of DevOps Usage in Practice," in Proc. IEEE Int. Conf. Softw. Maint. Evol. (ICSME), 2017, pp. 1-10.